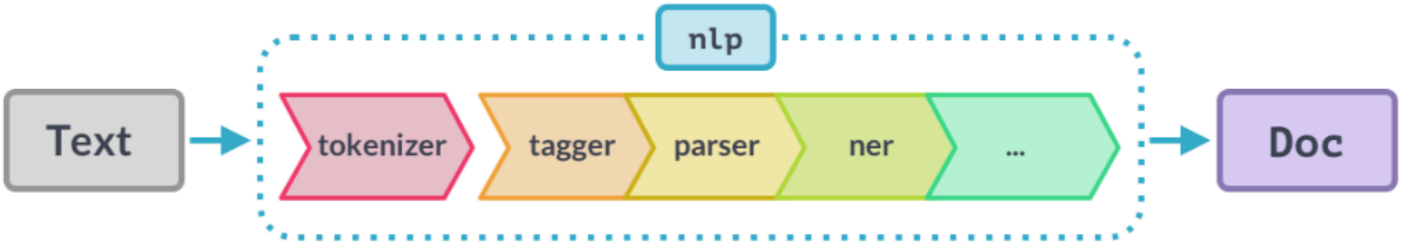
What happens when you call nlp?




```
doc = nlp("This is a sentence.")
```

Name	Description	Creates
tagger	Part-of-speech tagger	`Token.tag`, `Token.pos`
parser	Dependency parser	`Token.dep`, `Token.head`, `Doc.sents`, `Doc.noun_chunks`
ner	Named entity recognizer	`Doc.ents`, `Token.ent_iob`, `Token.ent_type`
textcat	Text classifier	`Doc.cats`

```
import spacy

# Load the en_core_web_sm pipeline
nlp = spacy.load('en_core_web_sm')

# Print the names of the pipeline components
print(nlp.pipe_names)

# Print the full pipeline of (name, component) tuples
print(nlp.pipeline)
```

Run code Submit

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec object at 0x7d00311f91e0>), ('tagge
r', <spacy.pipeline.tagger.Tagger object at 0x7d00311f9180>), ('parser', <spacy.
pipeline.dep_parser.DependencyParser object at 0x7d0031585380>), ('attribute_rul
er', <spacy.pipeline.attributeruler.AttributeRuler object at 0x7d00312057c0>),
('lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer object at 0x7d0049310
2c0>), ('ner', <spacy.pipeline.ner.EntityRecognizer object at 0x7d00315852a0>)]
```

```
@Language.component("custom_component")
def custom_component_function(doc):
    # Do something to the doc here
    return doc

nlp.add_pipe("custom_component")
```

Argument	Description	Example
`last`	If `True`, add last	`nlp.add_pipe("component", last=True)`
`first`	If `True`, add first	`nlp.add_pipe("component", first=True)`
`before`	Add before component	`nlp.add_pipe("component", before="ner")`
`after`	Add after component	`nlp.add_pipe("component", after="tagger")`

Setting custom attributes

- Add custom metadata to documents, tokens and spans
- Accessible via the `._` property

```
doc._.title = "My document"
token._.is_color = True
span._.has_color = False
```

- Registered on the global `Doc`, `Token` or `Span` using the `set_extension` method

```
# Import global classes
from spacy.tokens import Doc, Token, Span

# Set extensions on the Doc, Token and Span
Doc.set_extension("title", default=None)
Token.set_extension("is_color", default=False)
Span.set_extension("has_color", default=False)
```

```
# Define getter function
def get_is_color(token):
    colors = ["red", "yellow", "blue"]
    return token.text in colors

# Set extension on the Token with getter
Token.set_extension("is_color", getter=get_is_color)

doc = nlp("The sky is blue.")
print(doc[3]._.is_color, "-", doc[3].text)
```

```
True - blue
```

- `Span` extensions should almost always use a getter

```
from spacy.tokens import Span

# Define getter function
def get_has_color(span):
    colors = ["red", "yellow", "blue"]
    return any(token.text in colors for token in span)

# Set extension on the Span with getter
Span.set_extension("has_color", getter=get_has_color)

doc = nlp("The sky is blue.")
print(doc[1:4]._.has_color, "-", doc[1:4].text)
print(doc[0:2]._.has_color, "-", doc[0:2].text)
```

```
True - sky is blue
False - The sky
```

```
def has_token(doc, token_text):
    in_doc = token_text in [token.text for token in doc]
    return in_doc

# Set extension on the Doc with method
Doc.set_extension("has_token", method=has_token)

doc = nlp("The sky is blue.")
print(doc._.has_token("blue"), "- blue")
print(doc._.has_token("cloud"), "- cloud")
```

```
True - blue
False - cloud
```

```
def get_wikipedia_url(span):
    # Get a Wikipedia URL if the span has one of the labels
    if span.label_ in ("PERSON", "ORG", "GPE", "LOCATION"):
        entity_text = span.text.replace(" ", "_")
        return "https://en.wikipedia.org/w/index.php?search=" + entity_text

# Set the Span extension wikipedia_url using the getter get_wikipedia_url
Span.set_extension("wikipedia_url", getter=get_wikipedia_url)

doc = nlp(
    "In over fifty years from his very first recordings right through to his "
    "last album, David Bowie was at the vanguard of contemporary culture."
)
for ent in doc.ents:
    # Print the text and Wikipedia URL of the entity
    print(ent.text, ent._.wikipedia_url)
```

Run code Submit

```
 fifty years None
 first None
 David Bowie https://en.wikipedia.org/w/index.php?search=David_Bowie
```

```
with open("exercises/en/countries.json", encoding="utf8") as f:
    COUNTRIES = json.loads(f.read())

with open("exercises/en/capitals.json", encoding="utf8") as f:
    CAPITALS = json.loads(f.read())

nlp = spacy.blank("en")
matcher = PhraseMatcher(nlp.vocab)
matcher.add("COUNTRY", list(nlp.pipe(COUNTRIES)))

@Language.component("countries_component")
def countries_component_function(doc):
    # Create an entity Span with the label "GPE" for all matches
    matches = matcher(doc)
    doc.ents = [Span(doc, start, end, label="GPE") for match_id, start, end in matches]
    return doc

# Add the component to the pipeline
nlp.add_pipe("countries_component")
print(nlp.pipe_names)

# Getter that looks up the span text in the dictionary of country capitals
get_capital = lambda span: CAPITALS.get(span.text)

# Register the Span extension attribute "capital" with the getter get_capital
Span.set_extension("capital", getter=get_capital)

# Process the text and print the entity text, label and capital attributes
doc = nlp("Czech Republic may help Slovakia protect its airspace")
print([(ent.text, ent.label_, ent._.capital) for ent in doc.ents])
```

Run code Submit

OUTPUT

```
['countries_component']
[('Czech Republic', 'GPE', 'Prague'), ('Slovakia', 'GPE', 'Bratislava')]
```

Processing large volumes of text

- Use `nlp.pipe` method
- Processes texts as a stream, yields `Doc` objects
- Much faster than calling `nlp` on each text

BAD:

```
docs = [nlp(text) for text in LOTS_OF_TEXTS]
```

GOOD:

```
docs = list(nlp.pipe(LOTS_OF_TEXTS))
```

- Setting `as_tuples=True` on `nlp.pipe` lets you pass in `(text, context)` tuples
- Yields `(doc, context)` tuples
- Useful for associating metadata with the `doc`

```
data = [
    ("This is a text", {"id": 1, "page_number": 15}),
    ("And another text", {"id": 2, "page_number": 16}),
]

for doc, context in nlp.pipe(data, as_tuples=True):
    print(doc.text, context["page_number"])
```

```
This is a text 15
And another text 16
```

```
Doc.set_extension("id", default=None)
Doc.set_extension("page_number", default=None)

data = [
    ("This is a text", {"id": 1, "page_number": 15}),
    ("And another text", {"id": 2, "page_number": 16}),
]

for doc, context in nlp.pipe(data, as_tuples=True):
    doc._.id = context["id"]
    doc._.page_number = context["page_number"]
```

- Use `nlp.make_doc` to turn a text into a `Doc` object

BAD:

```
doc = nlp("Hello world")
```

GOOD:

```
doc = nlp.make_doc("Hello world!")
```

- Use `nlp.select_pipes` to temporarily disable one or more pipes

```
# Disable tagger and parser
with nlp.select_pipes(disable=["tagger", "parser"]):
    # Process the text and print the entities
    doc = nlp(text)
    print(doc.ents)
```

- Restores them after the `with` block
- Only runs the remaining components